

MODELO CONSOLIDADO · BASE DE DATOS

Sistema Cooperadora

Modelo entidad-relación unificado a partir de las dos versiones del equipo, con criterios de tipos de datos, normalización y script de creación.

DOCUMENTO	Modelo de datos consolidado — versión 3
PARTE DE	Sistcoop179 · módulo aportante e incremento 2
TABLAS	16
RELACIONES	28 claves foráneas
BASE	PostgreSQL
FECHA	10 de septiembre de 2026

CONTENIDO

- 1 · Qué se comparó y con qué criterio
- 2 · Equivalencias entre las tres versiones
- 3 · Qué se incorporó del otro modelo
- 4 · Qué se corrigió
- 5 · Criterios de tipos de datos
- 6 · Normalización aplicada
- 7 · Diagrama entidad-relación consolidado
- 8 · Pendientes del proyecto que este modelo resuelve
- 9 · Catálogo de tablas
- 10 · Reglas que el modelo hace cumplir
- Anexo A · Script de creación de la base
- Anexo B · Pruebas de las restricciones

1 Qué se comparó y con qué criterio

Este documento cruza dos modelos del mismo sistema: el del módulo aportante, hecho en dbdiagram, y el relevado del código actual. Las dos versiones resuelven el mismo problema y coinciden en casi todo el núcleo; se diferencian en dónde ponen el comprobante, el recibo y la validación, y en cómo tratan la distinción entre socio y no socio.

El criterio para elegir entre una y otra fue siempre el mismo: **que la base de datos por sí sola impida los errores**, sin depender de que el código los evite. Un dato repetido en dos tablas se puede desincronizar; una restricción declarada en el motor, no. Donde las dos versiones eran igual de correctas, se eligió la más simple.

El resultado son 16 tablas. La aplicación puede cambiar; lo que se fija acá es la estructura sobre la que va a apoyarse.

2 Equivalencias entre las tres versiones

Dónde termina cada tabla de cada modelo. Un guion significa que esa versión no tenía esa parte.

MODELO DEL MÓDULO APORTANTE	MODELO RELEVADO DEL CÓDIGO	MODELO CONSOLIDADO
EJERCICIOS	ejercicios	ejercicios
APORTANTES + TIPO_APORTANTES + FORMULARIO_SOCIO + FORMULARIO_OTROS	aportantes	aportantes (una sola tabla, sin subtipos)
SALDO_PENDIENTE	saldos_aportantes	saldos_aportantes
PAGOS	pagos	pagos
PAGOS_GRUPALES	pagos_grupales	pagos_grupales
COMPROBANTES + COMPROBANTES_OTROS	columnas dentro de pagos	operaciones
VALIDACIONES_PAGOS + VALIDACION_PAGOS_GRUPALES + ESTADOS_PAGO	columnas verificado_por_id y fecha_verificacion	verificaciones
RECIBOS + LETRAS	columnas numero_recibo y serie_recibo	recibos
CARRERAS	carreras	carreras
ANIO	carreras.anios (texto "1°,2°,3°")	carrera_anios
TIPO_APORTE	pagos.tipo	pagos.tipo con CHECK
—	usuarios	usuarios
—	fondos, movimientos_fondo, solicitudes_fondo	igual (fuera del módulo aportante)
—	auditoria	auditoria
—	saldos_aportantes.libreta_entregada	libretas (tabla propia)

3 Qué se incorporó del otro modelo

Cuatro decisiones del modelo del módulo aportante son mejores que las del modelo relevado, y pasaron al consolidado.

QUÉ SE TOMA	POR QUÉ
El comprobante como entidad propia	En tu modelo el comprobante eran cuatro columnas dentro de pagos. Tenerlo como tabla es lo correcto, y es lo que en el modelo consolidado se llama operaciones.
El historial de validaciones	VALIDACIONES_PAGOS guarda cada validación con su usuario y su fecha, en vez de pisar el dato anterior. Así queda registrado quién rechazó un pago antes de que otro lo aprobara.
El recibo como entidad con numeración propia	Un recibo es un comprobante formal con su letra y su número correlativo; eso no entra en dos columnas sueltas dentro de pagos.
La idea de separar el formulario del pago	Los dos formularios (socio y otros) apuntaban a algo real: no es lo mismo pagar la cuota que hacer un aporte adicional. En el consolidado esa diferencia vive en pagos.tipo, que es donde corresponde, en vez de partir a la persona en dos tablas.

4 Qué se corrigió

Estas observaciones son sobre el modelo del módulo aportante. Varias son de las que no se notan hasta que la base ya tiene datos cargados, así que conviene resolverlas ahora.

#	QUÉ PASA	DETALLE
1	Referencia colgada	VALIDACIONES_PAGOS y VALIDACION_PAGOS_GRUPALES tienen ID_USUARIO, pero no existe una tabla USUARIOS en el diagrama. La clave foránea no puede crearse.
2	Importes en int	IMPORTE, CUOTA, SALDO_PENDIENTE e IMPORTE_TOTAL son int. Una cuota de \$15.000,50 se guarda como 15000. En un sistema contable eso es un error que no se recupera.
3	DNI en int	Un DNI no es una cantidad con la que se opere: no se suma ni se promedia. Como int pierde ceros a la izquierda y no admite formatos.
4	Tipos incompatibles entre clave y referencia	TIPO_APORTANTES.ID_TIPO_APORTE es varchar y apunta a TIPO_APORTE.ID_TIPO_APORTE, que es int. La base rechaza esa clave foránea.
5	Identidad repartida en dos tablas	El nombre, el apellido y el DNI viven en FORMULARIO_SOCIO o en FORMULARIO_OTROS, no en APORTANTES. La misma persona puede quedar cargada como socio y como no socio con el mismo DNI, porque no hay una unicidad que abarque las dos tablas.
6	Arco exclusivo sin restricción	TIPO_APORTANTES tiene ID_FORM_SOCIO e ID_FORM_OTROS, los dos opcionales. Nada impide que estén los dos cargados, o ninguno.
7	Comprobante duplicado	COMPROBANTES y COMPROBANTES_OTROS tienen exactamente los mismos campos. Son dos tablas para el mismo concepto, arrastradas por la división socio / no socio.
8	El comprobante cuelga de la persona	FORMULARIO_SOCIO.ID_COMPROBANTE implica un comprobante por persona. Pero una persona hace muchos pagos, y cada pago tiene el suyo: el comprobante pertenece al pago.
9	Recibos sin vínculo al pago	RECIBOS no tiene clave foránea a PAGOS, así que no se puede saber a qué cobro corresponde un recibo emitido.
10	El estado, en dos lugares a la vez	PAGOS.ESTADO es un varchar libre y además existe la tabla ESTADOS_PAGO. Conviven dos fuentes para el mismo dato y pueden decir cosas distintas.
11	El pago grupal no puede repartir	PAGOS_GRUPALES apunta a un único ID_APORTANTE y no hay tabla de detalle. Una transferencia que cubre a cinco familias no se puede representar.
12	Fecha de aporte en la persona	APORTANTES.FECHA_APORTE guarda una sola fecha por persona, cuando cada pago tiene la suya. El dato está en la tabla equivocada.
13	Un año por carrera	CARRERAS.ID_ANIO apunta a la tabla ANIO, o sea que una carrera tiene un solo año. Es al revés: una carrera se cursa en varios años.
14	Nomenclatura mezclada	Conviven plural y singular (TIPO_APORTANTES / TIPO_APORTE), mayúsculas y minúsculas (FECHA / fecha_validacion). No rompe nada, pero un criterio único se lee mucho mejor y evita errores al escribir consultas.

5 Criterios de tipos de datos

El tipo de una columna no es un detalle de implementación: es la primera restricción de integridad que tiene el dato. Un importe guardado como `int` pierde los centavos para siempre, y no hay forma de recuperarlos después. Estos son los criterios aplicados en todo el modelo.

DATO	TIPO	POR QUÉ
Importes de dinero	<code>NUMERIC (12 , 2)</code>	Nunca <code>int</code> , porque pierde los centavos. Nunca <code>float</code> ni <code>real</code> : guardan en binario y <code>0,1 + 0,2</code> no da exactamente <code>0,3</code> , así que los totales terminan con diferencias de un centavo que nadie puede explicar.
DNI	<code>VARCHAR (15)</code>	No es una cantidad: no se suma ni se promedia. Como texto conserva los ceros a la izquierda y admite el formato que venga, y se normaliza al guardarlo.
CUIT	<code>VARCHAR (13)</code>	Mismo criterio que el DNI, con los guiones opcionales.
Año (de ejercicio o de cursada)	<code>SMALLINT</code>	Acá sí es un número: se ordena y se compara. <code>SMALLINT</code> alcanza y sobra.
Fecha del pago o de la asamblea	<code>DATE</code>	Es un día, no un instante. Guardar la hora invita a comparaciones que fallan por el horario.
Momentos del sistema	<code>TIMESTAMP</code>	<code>created_at</code> , la fecha de una verificación o la de una carga sí necesitan hora: sirven para reconstruir el orden de los hechos.
Estados y tipos	<code>VARCHAR (20) + CHECK</code>	El <code>CHECK</code> deja la lista de valores válidos declarada en la base. Una tabla de referencia también sirve, pero suma un <code>JOIN</code> a cada consulta; conviene cuando la lista la administra el usuario, no cuando la fija el negocio.
Banderas (activo, cerrado)	<code>BOOLEAN</code>	No <code>int 0/1</code> : el motor ya tiene el tipo y evita que aparezca un 2.
Códigos y hashes	<code>VARCHAR + UNIQUE</code>	El código de transacción y el hash del comprobante son identificadores externos: van como texto y con unicidad declarada.
Textos largos	<code>TEXT</code>	Justificaciones, observaciones y motivos de rechazo no tienen un largo previsible.
Claves primarias	<code>INTEGER (SERIAL)</code>	Una clave sustituta, numérica y sin significado de negocio. El DNI es único, pero no se usa como clave primaria: si mañana se corrige un DNI mal cargado hay que actualizar todas las tablas que lo referencian.

6 Normalización aplicada

Cada forma normal, dicha sobre este modelo y no en abstracto.

FORMA	QUÉ EXIGE	CÓMO SE CUMPLE ACÁ
1FN	Cada celda guarda un solo valor	carreras.anios guardaba "1°,2°,3°" en una sola columna: eso es un grupo repetitivo y viola la primera forma normal. En el consolidado pasa a la tabla carrera_anios, con una fila por año. Además permite declarar la clave foránea compuesta (carrera_id, anio) desde aportantes, que impide asignar a alguien un año que esa carrera no dicta.
2FN	Ningún atributo depende de una parte de la clave	La única clave compuesta del modelo es la de carrera_anios, que no tiene atributos propios. Las demás tablas usan clave sustituta, así que la segunda forma normal se cumple por construcción.
3FN	Ningún atributo depende de otro atributo no clave	Los datos derivados están identificados y aislados: fondos.saldo se reconstruye desde movimientos_fondo, saldos_aportantes desde los pagos verificados, y pagos.estado desde la última verificación. Se conservan por rendimiento y se declaran como caché, no como fuente de verdad.
BCNF	Toda dependencia funcional sale de una clave	aportantes.dni es único, así que determina toda la fila y es una clave candidata legítima. No hay determinantes que no sean claves.
4FN	Sin dependencias multivaluadas independientes	No hay ninguna tabla que cruce dos listas que no tengan relación entre sí. carrera_anios relaciona una carrera con sus años y nada más.
5FN	Sin uniones que se puedan descomponer	La relación entre pago, aportante y ejercicio es funcional: un pago tiene exactamente un aportante y un ejercicio. No es una relación ternaria descomponible.
Excepción documentada	La copia fija del recibo	recibos guarda el importe, el documento y la razón social del momento de la emisión, aunque esos datos ya estén en pagos y en aportantes. No es redundancia por descuido: un comprobante formal tiene que seguir diciéndolo mismo dentro de cinco años, aunque la persona cambie de apellido o se corrija el importe de un pago.

7 Diagrama entidad-relación consolidado

El diagrama de la página siguiente se lee de izquierda a derecha: primero las tablas que no dependen de nadie, después las que las referencian. Se entrega además como imagen aparte, en alta resolución, para imprimirlo en A3 o mirarlo con zoom.

La notación es pata de gallo: **|** exactamente uno, **|o** cero o uno, **o{** cero o muchos. El color de la cabecera indica el área: **verde** núcleo institucional, **terracota** aportantes y cobros, **verde azulado** fondos y solicitudes, **mostaza** trazabilidad. Cada flecha llega exactamente al campo que la implementa, y toma el color del área de la que sale: donde varias líneas corren juntas, el color dice de dónde viene cada una.

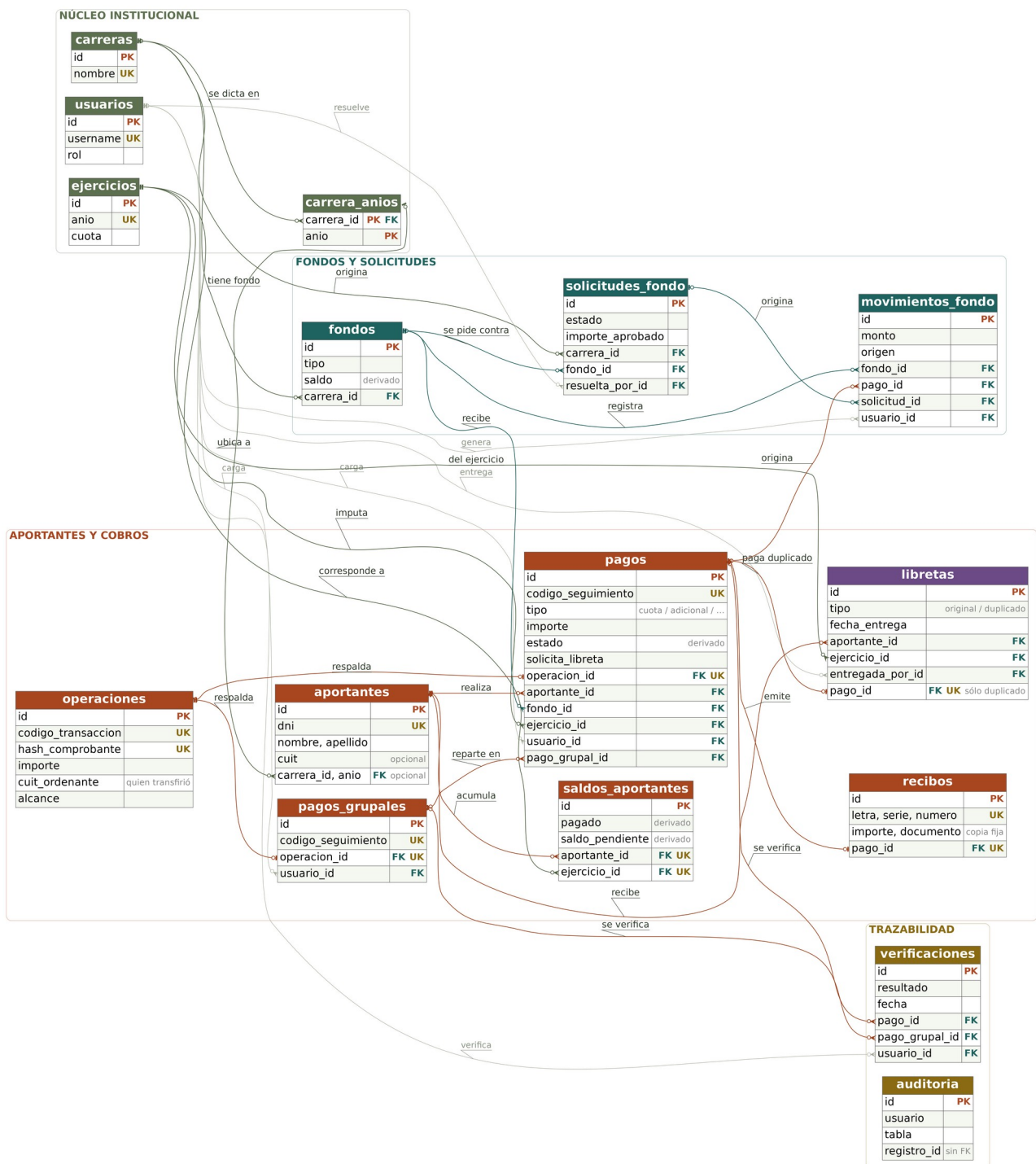


Figura 1 — Modelo entidad-relación consolidado del Sistema Cooperadora (16 tablas, 28 claves foráneas).

8 Pendientes del proyecto que este modelo resuelve

El documento docs/PENDIENTES.md del repositorio anota siete cuestiones abiertas. Cuatro de ellas son del modelo de datos, y el consolidado las cierra.

PEND.	QUÉ DECÍA	CÓMO QUEDA
5	carreras.anios guarda "1º,2º,3º" como texto y aportantes.anio es texto libre, sin ninguna restricción. El propio documento propone una tabla carreras_anios.	Resuelto. carrera_anios existe con una fila por año, y aportantes referencia (carrera_id, anio) con clave foránea compuesta: la base impide un año que esa carrera no dicte.
4	pagos_grupales_detalle.estado sugiere que se puede verificar a una persona del grupo por separado, pero el código siempre verifica la transferencia entera.	Resuelto por eliminación. Al no existir la tabla de detalle, cada línea del reparto es un pago con su propio estado: la verificación parcial queda posible si algún día se define, y mientras tanto no hay una columna que prometa algo que el sistema no hace.
3	pagos.cuit: nunca se definió si es del aportante o de un tercero que transfirió por él.	Separado en dos columnas distintas, que es lo que la ambigüedad pedía: aportantes.cuit cuando el aportante mismo es una empresa, y operaciones.cuit_ordenante cuando quien puso la plata es otro. Esta última es la que sirve para conciliar contra el extracto.
1	Las migraciones se probaron sólo contra SQLite; falta correrlas contra un PostgreSQL vacío.	Hecho para este modelo: el script del anexo A se ejecutó sobre PostgreSQL 16 vacío y las 16 tablas, sus claves, sus índices parciales y sus CHECK se crearon sin errores.

Los tres restantes (el límite de peticiones por IP con varios workers, la variable MAX_CONTENT_LENGTH que no se lee, y el control de fecha_asamblea) no son del modelo de datos y quedan fuera de este documento.

9 Catálogo de tablas

Las marcas de cada campo: **PK** clave primaria, **FK** clave foránea, **UK** restricción de unicidad. Las notas **en color** señalan lo que cambia respecto del modelo relevado.

NÚCLEO INSTITUCIONAL

usuarios

Personal de la Cooperadora que carga, verifica o resuelve.

Faltaba en el modelo del compañero, aunque varias tablas la referenciaban.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
username UK	VARCHAR(50)	no nulo
email UK	VARCHAR(120)	no nulo
password_hash	VARCHAR(255)	hash, nunca texto plano
nombre, apellido	VARCHAR(100)	no nulo
rol	VARCHAR(20)	CHECK admin / asistente / tesorera / preceptoria
activo	BOOLEAN	por defecto true
ultimo_login	TIMESTAMP	—
created_at, updated_at	TIMESTAMP	—

carreras

Cada carrera de la institución.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
nombre UK	VARCHAR(100)	no nulo, no se repite
activo	BOOLEAN	por defecto true
created_at	TIMESTAMP	—

carrera_anios **NUEVA**

Los años que se dictan en cada carrera: una fila por año.

Reemplaza la columna de texto "1°,2°,3°", que violaba la primera forma normal. Su clave compuesta permite validar, desde aportantes, que el año exista en esa carrera.

CAMPO	TIPO	NOTAS
carrera_id PK FK	INTEGER → carreras.id	parte de la clave primaria
anio PK	SMALLINT	CHECK entre 1 y 7

ejercicios

Año contable con el importe de cuota fijado en asamblea.

Índice único parcial: sólo un ejercicio vigente (activo y no cerrado) a la vez.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
anio UK	SMALLINT	no nulo

CAMPO	TIPO	NOTAS
cuota	NUMERIC(12,2)	no nulo, CHECK > 0
fecha_asamblea	DATE	—
activo, cerrado	BOOLEAN	definen cuál es el vigente
created_at	TIMESTAMP	—

APORTANTES

aportantes MODIFICADA

Quien le transfiere plata a la Cooperadora. Una sola tabla, sin subtipos.

La diferencia entre pagar la cuota y hacer un aporte adicional es del PAGO, no de la persona: la misma persona puede hacer las dos cosas. Por eso no hay tipo acá, y el DNI es único en todo el sistema.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
dni UK	VARCHAR(15)	no nulo, se normaliza sin puntos
nombre, apellido	VARCHAR(100)	no nulo
cuit	VARCHAR(13)	opcional — cuando el aportante mismo es una empresa
carrera_id, anio FK	→ carrera_anios	FK compuesta, opcionales
email, telefono	VARCHAR(120)	—
activo	BOOLEAN	por defecto true
created_at, updated_at	TIMESTAMP	—

ENTRADA DE DINERO

operaciones NUEVA

Cada movimiento de dinero que entra: una transferencia, un depósito o una entrega en efectivo.

Es la única tabla dueña del código de transacción y del hash del comprobante. Al vivir en un solo lugar, la misma transferencia no se puede cargar dos veces, ni siquiera desde otra pantalla.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
medio_pago	VARCHAR(20)	CHECK transferencia / deposito / efectivo
codigo_transaccion UK	VARCHAR(100)	nulo si es efectivo; los nulos no colisionan
hash_comprobante UK	VARCHAR(64)	SHA-256 del archivo
comprobante_nombre, comprobante_ruta	VARCHAR(255)	archivo servido bajo sesión
importe	NUMERIC(12,2)	no nulo, CHECK > 0
fecha	DATE	no nulo
cuit_ordenante	VARCHAR(13)	CUIT de quien transfirió, para conciliar con el banco
alcance	VARCHAR(12)	CHECK individual / grupal
created_at	TIMESTAMP	—

pagos MODIFICADA

Cada cobro imputado a un aportante. Nace pendiente y no mueve saldo hasta verificarse.

Un pago entra de una de dos formas y nunca de las dos: por su propia operación bancaria, o como parte del reparto de un pago grupal. Un CHECK obliga a que exactamente una de las dos referencias esté cargada.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
codigo_seguimiento UK	VARCHAR(20)	público, tipo SC-A3F9K2XY
tipo	VARCHAR(30)	CHECK cuota / adicional / aporte_carrera / libreta_duplicado
importe	NUMERIC(12,2)	no nulo, CHECK > 0
fecha	DATE	no nulo
estado	VARCHAR(20)	CHECK pendiente / verificado / rechazado / anulado — derivado de la última verificación
solicita_libreta	BOOLEAN	por defecto false
observaciones	TEXT	—
created_at, updated_at	TIMESTAMP	—
operacion_id FK UK	→ operaciones.id	nulo sólo si viene de un reparto grupal
aportante_id FK	→ aportantes.id	no nulo
fondo_id FK	→ fondos.id	no nulo
ejercicio_id FK	→ ejercicios.id	no nulo
usuario_id FK	→ usuarios.id	quien lo carga
pago_grupal_id FK	→ pagos_grupales.id	único junto con aportante_id

pagos_grupales MODIFICADA

Una sola operación bancaria que cubre a varias personas.

El reparto no tiene tabla propia: cada línea es una fila de pagos con pago_grupal_id cargado. Así el monto y el estado de cada parte viven en un solo lugar.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
codigo_seguimiento UK	VARCHAR(20)	—
operacion_id FK UK	→ operaciones.id	no nulo, alcance = grupal
estado	VARCHAR(20)	CHECK pendiente / verificado / rechazado
usuario_id FK	→ usuarios.id	quien lo carga
created_at	TIMESTAMP	—

verificaciones NUEVA

Cada vez que alguien revisa un pago contra el banco queda una fila, con su resultado.

Guarda el historial completo: si un pago se rechaza y después se aprueba, quedan las dos. El estado que muestra pagos.estado es el de la última verificación. Un CHECK obliga a que la fila apunte a un pago individual o a uno grupal, nunca a los dos.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
resultado	VARCHAR(20)	CHECK verificado / rechazado
motivo	TEXT	obligatorio si el resultado es rechazado
fecha	TIMESTAMP	no nulo

CAMPO	TIPO	NOTAS
pago_id FK	→ pagos.id	excluyente con pago_grupal_id
pago_grupal_id FK	→ pagos_grupales.id	excluyente con pago_id
usuario_id FK	→ usuarios.id	no nulo — quién verificó

recibos NUEVA

Comprobante formal emitido por un pago verificado, con su numeración correlativa.

Los datos del aportante y el importe se copian al emitirlo y no se vuelven a tocar: un comprobante tiene que seguir diciendo lo mismo dentro de cinco años.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
letra UK	CHAR(1)	CHECK A / B / C / X — parte de la unidad
serie UK	SMALLINT	punto de emisión
numero UK	INTEGER	correlativo; único junto con letra y serie
fecha_emision	TIMESTAMP	no nulo
concepto	VARCHAR(200)	—
razon_social, documento, importe	VARCHAR / NUMERIC(12,2)	copia fija del momento de la emisión
pago_id FK UK	→ pagos.id	no nulo — un recibo por pago

saldos_aportantes MODIFICADA

Cómo viene la cuota de una persona en un ejercicio. Tabla derivada, no fuente de verdad.

Se reconstruye sumando los pagos verificados. Perdió la columna estado (se deduce de saldo_pendiente) y perdió la entrega de la libreta, que pasó a tabla propia.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
aportante_id FK UK	→ aportantes.id	no nulo — único junto con ejercicio_id
ejercicio_id FK UK	→ ejercicios.id	no nulo
pagado	NUMERIC(12,2)	por defecto 0, sólo pagos verificados
saldo_pendiente	NUMERIC(12,2)	—
updated_at	TIMESTAMP	—

libretas NUEVA

Cada entrega de libreta: quién la recibió, de qué año, quién la entregó y cuándo.

Antes esto eran dos columnas dentro de saldos_aportantes, que es una caché reconstruible desde los pagos: un hecho real —se entregó tal día— no puede vivir en una tabla que se puede regenerar. Además ahora queda registrado quién la entregó, y se pueden registrar duplicados sin pisar la original.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
tipo	VARCHAR(12)	CHECK original / duplicado
fecha_entrega	TIMESTAMP	no nulo
observaciones	TEXT	—
aportante_id FK	→ aportantes.id	no nulo
ejercicio_id FK	→ ejercicios.id	no nulo

CAMPO	TIPO	NOTAS
entregada_por_id FK	→ usuarios.id	no nulo — quién la entregó
pago_id FK UK	→ pagos.id	obligatorio si es duplicado; el duplicado se cobra

FONDOS Y SOLICITUDES

fondos

Cada "bolsillo" de la Cooperadora: uno general y uno por carrera.

Único (carrera_id, tipo): una carrera no puede tener dos fondos propios del mismo tipo.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
nombre	VARCHAR(100)	no nulo
tipo	VARCHAR(20)	CHECK capital / carrera / evento
saldo	NUMERIC(14,2)	derivado de movimientos_fondo
activo	BOOLEAN	por defecto true
carrera_id FK	→ carreras.id	nulo en el fondo de capital
created_at, updated_at	TIMESTAMP	—

movimientos_fondo MODIFICADA

Cada vez que el saldo de un fondo cambia queda un renglón, y ahora también queda por qué.

Un CHECK obliga a que la clave foránea cargada coincida con el origen declarado. No se edita ni se borra: una corrección se hace con un movimiento al revés.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
monto	NUMERIC(14,2)	no nulo, negativo si el saldo baja
saldo_anterior, saldo_resultante	NUMERIC(14,2)	el resultante es no nulo
motivo	VARCHAR(150)	sólo descriptivo
origen	VARCHAR(20)	CHECK pago / solicitud / ajuste / apertura
fondo_id FK	→ fondos.id	no nulo
pago_id FK	→ pagos.id	obligatorio si origen = pago
solicitud_id FK	→ solicitudes_fondo.id	obligatorio si origen = solicitud
usuario_id FK	→ usuarios.id	quién lo generó
created_at	TIMESTAMP	—

solicitudes_fondo MODIFICADA

Pedido de fondos, evento o viaje que presenta un curso o un docente.

Al aprobarse queda registrado cuánto se aprobó y contra qué fondo; el egreso se asienta en movimientos_fondo.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
codigo_seguimiento UK	VARCHAR(20)	—
responsable, contacto	VARCHAR(120)	no nulo
curso	VARCHAR(50)	—

CAMPO	TIPO	NOTAS
tipo	VARCHAR(20)	CHECK fondos / evento / viaje
concepto	VARCHAR(200)	no nulo
importe_estimado	NUMERIC(14,2)	lo que pide el solicitante
importe_aprobado	NUMERIC(14,2)	lo que efectivamente se otorga
fecha_estimada	DATE	—
justificacion	TEXT	no nulo
estado	VARCHAR(20)	CHECK pendiente / aprobada / rechazada
resolucion	TEXT	—
fecha_resolucion	TIMESTAMP	—
carrera_id FK	→ carreras.id	opcional
fondo_id FK	→ fondos.id	de qué fondo sale la plata
resuelta_por_id FK	→ usuarios.id	opcional

TRAZABILIDAD

auditoria

Un renglón por cada operación importante del sistema. No se edita ni se borra.

Deliberadamente sin claves foráneas: guarda usuario y tabla/registro_id como texto, para seguir existiendo aunque el registro original se modifique.

CAMPO	TIPO	NOTAS
id PK	SERIAL	—
usuario	VARCHAR(50)	no nulo, texto libre
accion	VARCHAR(100)	no nulo
tabla	VARCHAR(50)	nombre de la tabla afectada
registro_id	INTEGER	sin FK real
detalle	JSONB	—
ip, user_agent	VARCHAR(255)	—
created_at	TIMESTAMP	—

10 Reglas que el modelo hace cumplir

Ninguna de estas reglas depende del código de la aplicación: todas están declaradas en la base y se pueden mostrar en el script del anexo.

REGLA	QUÉ GARANTIZA
Identidad única	Una persona existe una sola vez: el DNI es único en aportantes, sea socio o no socio.
Socio o no socio	Al no socio se le exige CUIT; al socio, carrera y año. Los dos casos son excluyentes y están declarados con CHECK.
Año que existe	A un socio no se le puede asignar un año que su carrera no dicta: la clave foránea compuesta (carrera_id, anio) lo impide.
Comprobante único	Una transferencia se carga una sola vez en todo el sistema: el código de transacción y el hash viven únicamente en operaciones.
Origen del pago	Un pago entra por su propia operación bancaria o por el reparto de un pago grupal, nunca por las dos vías ni por ninguna.
Reparto sin repetidos	Un aportante no puede aparecer dos veces dentro del mismo pago grupal.
Pago verificado	Un pago nace pendiente y no mueve ningún saldo hasta que se lo verifica contra el banco.
Historial de verificaciones	Cada revisión deja su fila con usuario, fecha y resultado. Un rechazo seguido de una aprobación queda registrado como dos hechos, no como uno pisando al otro.
Rechazo con motivo	Una verificación con resultado rechazado tiene que explicar por qué.
Recibo por pago	Un pago emite a lo sumo un recibo, y la combinación letra + serie + número no se repite.
Ejercicio obligatorio	Todo pago se imputa a un ejercicio, así que siempre puede reflejarse en el saldo del aportante.
Importes positivos	Los importes de operaciones, pagos y cuotas son mayores que cero, y se guardan con dos decimales.
Trazabilidad del fondo	Todo movimiento de fondo declara su origen, y la clave foránea cargada tiene que coincidir con ese origen.
Solicitud aprobada	Una solicitud aprobada tiene importe aprobado y fondo asignado; el egreso queda asentado como movimiento con origen = solicitud.
Fondo por carrera	Una carrera tiene a lo sumo un fondo propio por tipo.
Un solo vigente	Sólo puede haber un fondo de capital activo, y un ejercicio vigente, a la vez.
Cuota congelada	La cuota de un ejercicio no se puede modificar una vez que ya tiene pagos cargados.
Cachés reconstruibles	fondos.saldo, saldos_aptantes y pagos.estado son datos derivados: se reconstruyen desde las tablas de hechos.
Una libreta por año	Una persona recibe una sola libreta original por ejercicio. Los duplicados no se limitan, pero cada uno tiene que tener su pago.
Duplicado con pago	Una entrega de tipo duplicado exige el pago que la respalda. La original no exige nada: la cuota es voluntaria y el sistema no retiene documentación (RF-13).
Entrega con responsable	Toda entrega de libreta registra qué usuario la hizo y cuándo.
Auditoría inmutable	auditoria es de sólo inserción, sin claves foráneas, para sobrevivir a cambios en los datos originales.

A Anexo · Script de creación de la base

Escrito para PostgreSQL. Las tablas están en orden de dependencia, así que el archivo se puede correr de arriba a abajo sin reordenar nada. Si el proyecto usa Alembic, cada bloque puede ser una revisión.

Los comentarios marcan las restricciones que resuelven cada una de las reglas de la sección 10.

```
-- =====
-- Sistema Cooperadora – modelo consolidado
-- PostgreSQL. Las tablas están en orden de dependencia:
-- se puede correr el archivo de arriba a abajo.
-- =====

-- ----- 1 · Núcleo institucional -----

CREATE TABLE usuarios (
    id            SERIAL          PRIMARY KEY,
    username      VARCHAR(50)    NOT NULL UNIQUE,
    email         VARCHAR(120)   NOT NULL UNIQUE,
    password_hash VARCHAR(255)   NOT NULL,
    nombre        VARCHAR(100)   NOT NULL,
    apellido      VARCHAR(100)   NOT NULL,
    rol           VARCHAR(20)     NOT NULL,
    activo        BOOLEAN        NOT NULL DEFAULT TRUE,
    ultimo_login  TIMESTAMP,
    created_at    TIMESTAMP      NOT NULL DEFAULT now(),
    updated_at    TIMESTAMP      NOT NULL DEFAULT now(),
    CONSTRAINT ck_usuarios_rol
        CHECK (rol IN ('admin','asistente','tesorera','preceptoria'))
);

CREATE TABLE carreras (
    id            SERIAL          PRIMARY KEY,
    nombre        VARCHAR(100)   NOT NULL UNIQUE,
    activo        BOOLEAN        NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMP      NOT NULL DEFAULT now()
);

-- Una fila por año dictado. Reemplaza la columna de texto "1°,2°,3°".
CREATE TABLE carrera_anios (
    carrera_id INTEGER NOT NULL REFERENCES carreras(id),
    anio        SMALLINT NOT NULL,
    PRIMARY KEY (carrera_id, anio),
    CONSTRAINT ck_carrera_anios_rango CHECK (anio BETWEEN 1 AND 7)
);

CREATE TABLE ejercicios (
    id            SERIAL          PRIMARY KEY,
    anio          SMALLINT        NOT NULL UNIQUE,
    cuota         NUMERIC(12,2)  NOT NULL,
    fecha_asamblea DATE,
    activo        BOOLEAN        NOT NULL DEFAULT TRUE,
    cerrado       BOOLEAN        NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMP      NOT NULL DEFAULT now(),
    CONSTRAINT ck_ejercicios_cuota CHECK (cuota > 0)
);

-- Sólo puede haber un ejercicio vigente a la vez.
CREATE UNIQUE INDEX uq_ejercicio_vigente
    ON ejercicios ((TRUE)) WHERE activo AND NOT cerrado;

-- ----- 2 · Aportantes -----
-- Una sola tabla: la persona es la misma sin importar qué formulario
-- llene. Lo que cambia entre "cuota" y "aporte adicional" es el PAGO,
-- no el aportante: eso vive en pagos.tipo.

CREATE TABLE aportantes (
    id            SERIAL          PRIMARY KEY,
```

```

dni          VARCHAR(15) NOT NULL UNIQUE,
nombre       VARCHAR(100) NOT NULL,
apellido     VARCHAR(100) NOT NULL,
cuit         VARCHAR(13),
carrera_id   INTEGER,
anio         SMALLINT,
email        VARCHAR(120),
telefono     VARCHAR(30),
activo       BOOLEAN      NOT NULL DEFAULT TRUE,
created_at   TIMESTAMP    NOT NULL DEFAULT now(),
updated_at   TIMESTAMP    NOT NULL DEFAULT now(),
-- carrera y año son opcionales: quien hace un aporte adicional puede no
-- estar ligado a ninguna carrera. Si se cargan, el año tiene que existir
-- en esa carrera.
CONSTRAINT fk_aportantes_carrera_anio
    FOREIGN KEY (carrera_id, anio) REFERENCES carreras(carrera_id, anio)
);

-- ----- 3 · Entrada de dinero -----

-- Única dueña del código de transacción y del hash del comprobante.
CREATE TABLE operaciones (
    id          SERIAL      PRIMARY KEY,
    medio_pago   VARCHAR(20) NOT NULL,
    codigo_transaccion VARCHAR(100) UNIQUE,
    hash_comprobante VARCHAR(64) UNIQUE,
    comprobante_nombre VARCHAR(255),
    comprobante_ruta   VARCHAR(255),
    importe       NUMERIC(12,2) NOT NULL,
    fecha         DATE       NOT NULL,
    -- CUIT de quien efectivamente hizo la transferencia. Va acá y no en el
    -- pago porque sirve para conciliar contra el extracto del banco: quien
    -- pone la plata puede no ser el aportante (una empresa, un familiar).
    cuit_ordenante VARCHAR(13),
    alcance       VARCHAR(12) NOT NULL,
    created_at    TIMESTAMP  NOT NULL DEFAULT now(),
    CONSTRAINT ck_operaciones_medio CHECK (medio_pago IN ('transferencia','deposito','efectivo')),
    CONSTRAINT ck_operaciones_alcance CHECK (alcance IN ('individual','gruppal')),
    CONSTRAINT ck_operaciones_importe CHECK (importe > 0),
    -- una transferencia siempre trae código; el efectivo, no
    CONSTRAINT ck_operaciones_codigo CHECK (
        medio_pago = 'efectivo' OR codigo_transaccion IS NOT NULL)
);

CREATE TABLE fondos (
    id          SERIAL      PRIMARY KEY,
    nombre      VARCHAR(100) NOT NULL,
    tipo        VARCHAR(20) NOT NULL,
    saldo       NUMERIC(14,2) NOT NULL DEFAULT 0,
    activo      BOOLEAN      NOT NULL DEFAULT TRUE,
    carrera_id  INTEGER      REFERENCES carreras(id),
    created_at  TIMESTAMP    NOT NULL DEFAULT now(),
    updated_at  TIMESTAMP    NOT NULL DEFAULT now(),
    CONSTRAINT ck_fondos_tipo CHECK (tipo IN ('capital','carrera','evento')),
    CONSTRAINT uq_fondos_carrera_tipo UNIQUE (carrera_id, tipo)
);

CREATE TABLE pagos_grupales (
    id          SERIAL      PRIMARY KEY,
    codigo_seguimiento VARCHAR(20) NOT NULL UNIQUE,
    operacion_id INTEGER    NOT NULL UNIQUE REFERENCES operaciones(id),
    estado      VARCHAR(20) NOT NULL DEFAULT 'pendiente',
    usuario_id  INTEGER      REFERENCES usuarios(id),
    created_at  TIMESTAMP    NOT NULL DEFAULT now(),
    CONSTRAINT ck_pg_estado CHECK (estado IN ('pendiente','verificado','rechazado'))
);

CREATE TABLE solicitudes_fondo (
    id          SERIAL      PRIMARY KEY,

```

```

codigo_seguimiento VARCHAR(20) NOT NULL UNIQUE,
responsable        VARCHAR(120) NOT NULL,
contacto           VARCHAR(120) NOT NULL,
curso              VARCHAR(50),
tipo               VARCHAR(20) NOT NULL,
concepto           VARCHAR(200) NOT NULL,
importe_estimado   NUMERIC(14,2),
importe_aprobado   NUMERIC(14,2),
fecha_estimada     DATE,
justificacion      TEXT NOT NULL,
estado             VARCHAR(20) NOT NULL DEFAULT 'pendiente',
resolucion         TEXT,
fecha_resolucion   TIMESTAMP,
carrera_id         INTEGER REFERENCES carreras(id),
fondo_id           INTEGER REFERENCES fondos(id),
resuelta_por_id    INTEGER REFERENCES usuarios(id),
created_at         TIMESTAMP NOT NULL DEFAULT now(),
CONSTRAINT ck_sol_tipo CHECK (tipo IN ('fondos','evento','viaje')),
CONSTRAINT ck_sol_estado CHECK (estado IN ('pendiente','aprobada','rechazada')),
-- si está aprobada, tiene que decir cuánto y contra qué fondo
CONSTRAINT ck_sol_aprobada CHECK (
    estado <> 'aprobada'
    OR (importe_aprobado IS NOT NULL AND fondo_id IS NOT NULL))
);

CREATE TABLE pagos (
    id SERIAL PRIMARY KEY,
    codigo_seguimiento VARCHAR(20) NOT NULL UNIQUE,
    tipo VARCHAR(30) NOT NULL,
    importe NUMERIC(12,2) NOT NULL,
    fecha DATE NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'pendiente',
    solicita_libreta BOOLEAN NOT NULL DEFAULT FALSE,
    observaciones TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    updated_at TIMESTAMP NOT NULL DEFAULT now(),
    operacion_id INTEGER UNIQUE REFERENCES operaciones(id),
    aportante_id INTEGER NOT NULL REFERENCES aportantes(id),
    fondo_id INTEGER NOT NULL REFERENCES fondos(id),
    ejercicio_id INTEGER NOT NULL REFERENCES ejercicios(id),
    usuario_id INTEGER REFERENCES usuarios(id),
    pago_grupal_id INTEGER REFERENCES pagos_grupales(id),
    -- 'cuota_grupal' sale de la lista: un pago de cuota que viene de un
    -- reparto se reconoce porque tiene pago_grupal_id cargado.
    CONSTRAINT ck_pagos_tipo CHECK (tipo IN ('cuota','adicional','aporte_carrera','libreta_duplicado')),
    CONSTRAINT ck_pagos_estado CHECK (estado IN ('pendiente','verificado','rechazado','anulado')),
    CONSTRAINT ck_pagos_importe CHECK (importe > 0),
    -- o entra por su propia operación bancaria, o viene de un reparto grupal
    CONSTRAINT ck_pagos_origen CHECK (
        (operacion_id IS NOT NULL AND pago_grupal_id IS NULL)
        OR (operacion_id IS NULL AND pago_grupal_id IS NOT NULL)),
    -- un aportante no puede repetirse dentro del mismo pago grupal
    CONSTRAINT uq_pagos_grupal_aportante UNIQUE (pago_grupal_id, aportante_id)
);

-- ----- 4 - Verificación y recibos -----

CREATE TABLE verificaciones (
    id SERIAL PRIMARY KEY,
    resultado VARCHAR(20) NOT NULL,
    motivo TEXT,
    fecha TIMESTAMP NOT NULL DEFAULT now(),
    pago_id INTEGER REFERENCES pagos(id),
    pago_grupal_id INTEGER REFERENCES pagos_grupales(id),
    usuario_id INTEGER NOT NULL REFERENCES usuarios(id),
    CONSTRAINT ck_ver_resultado CHECK (resultado IN ('verificado','rechazado')),
    -- apunta a un pago individual o a uno grupal, nunca a los dos
    CONSTRAINT ck_ver_objeto CHECK (
        (pago_id IS NOT NULL AND pago_grupal_id IS NULL)

```

```

    OR (pago_id IS NULL      AND pago_grupal_id IS NOT NULL)),
    -- un rechazo tiene que explicarse
    CONSTRAINT ck_ver_motivo CHECK (
        resultado <> 'rechazado' OR motivo IS NOT NULL)
);

CREATE TABLE recibos (
    id            SERIAL          PRIMARY KEY,
    letra         CHAR(1)        NOT NULL,
    serie         SMALLINT       NOT NULL,
    numero        INTEGER        NOT NULL,
    fecha_emision TIMESTAMP      NOT NULL DEFAULT now(),
    concepto      VARCHAR(200),
    razon_social  VARCHAR(200)   NOT NULL,
    documento     VARCHAR(15)    NOT NULL,
    importe       NUMERIC(12,2)  NOT NULL,
    pago_id       INTEGER        NOT NULL UNIQUE REFERENCES pagos(id),
    CONSTRAINT ck_recibos_letra CHECK (letra IN ('A','B','C','X')),
    CONSTRAINT uq_recibos_numeracion UNIQUE (letra, serie, numero)
);

-- La entrega de la libreta NO vive en saldos_aportantes: esa tabla es una
-- caché que se puede reconstruir desde los pagos, y un hecho real (se
-- entregó, tal día, tal persona) no puede vivir en algo reconstruible.
CREATE TABLE libretas (
    id            SERIAL          PRIMARY KEY,
    tipo          VARCHAR(12)    NOT NULL,
    fecha_entrega TIMESTAMP      NOT NULL DEFAULT now(),
    observaciones TEXT,
    aportante_id  INTEGER        NOT NULL REFERENCES aportantes(id),
    ejercicio_id  INTEGER        NOT NULL REFERENCES ejercicios(id),
    entregada_por_id INTEGER      NOT NULL REFERENCES usuarios(id),
    -- el duplicado se cobra: apunta al pago de tipo libreta_duplicado
    pago_id       INTEGER        UNIQUE REFERENCES pagos(id),
    CONSTRAINT ck_libretas_tipo CHECK (tipo IN ('original','duplicado')),
    CONSTRAINT ck_libretas_duplicado CHECK (
        tipo <> 'duplicado' OR pago_id IS NOT NULL)
);

-- Una sola libreta original por persona y ejercicio. Los duplicados no se
-- limitan: cada uno tiene su pago.
CREATE UNIQUE INDEX uq_libreta_original
    ON libretas (aportante_id, ejercicio_id) WHERE tipo = 'original';

-- ----- 5 · Datos derivados -----

CREATE TABLE saldos_aportantes (
    id            SERIAL          PRIMARY KEY,
    aportante_id  INTEGER        NOT NULL REFERENCES aportantes(id),
    ejercicio_id  INTEGER        NOT NULL REFERENCES ejercicios(id),
    pagado        NUMERIC(12,2)  NOT NULL DEFAULT 0,
    saldo_pendiente NUMERIC(12,2) NOT NULL DEFAULT 0,
    updated_at    TIMESTAMP      NOT NULL DEFAULT now(),
    CONSTRAINT uq_saldos_aportante_ejercicio UNIQUE (aportante_id, ejercicio_id)
);

CREATE TABLE movimientos_fondo (
    id            SERIAL          PRIMARY KEY,
    monto         NUMERIC(14,2)  NOT NULL,
    saldo_anterior NUMERIC(14,2),
    saldo_resultante NUMERIC(14,2) NOT NULL,
    motivo        VARCHAR(150),
    origen        VARCHAR(20)    NOT NULL,
    fondo_id      INTEGER        NOT NULL REFERENCES fondos(id),
    pago_id       INTEGER        REFERENCES pagos(id),
    solicitud_id  INTEGER        REFERENCES solicitudes_fondo(id),
    usuario_id    INTEGER        REFERENCES usuarios(id),
    created_at    TIMESTAMP      NOT NULL DEFAULT now(),
    CONSTRAINT ck_mov_origen CHECK (origen IN ('pago','solicitud','ajuste','apertura')),

```

```
-- la clave foránea cargada tiene que coincidir con el origen declarado
CONSTRAINT ck_mov_coherencia CHECK (
    (origen = 'pago'          AND pago_id          IS NOT NULL AND solicitud_id IS NULL)
    OR (origen = 'solicitud' AND solicitud_id IS NOT NULL AND pago_id          IS NULL)
    OR (origen IN ('ajuste','apertura') AND pago_id IS NULL AND solicitud_id IS NULL))
);
```

```
-- ----- 6 · Trazabilidad -----
```

```
CREATE TABLE auditoria (
    id          SERIAL          PRIMARY KEY,
    usuario     VARCHAR(50) NOT NULL,
    accion      VARCHAR(100) NOT NULL,
    tabla       VARCHAR(50),
    registro_id INTEGER,
    detalle     JSONB,
    ip          VARCHAR(45),
    user_agent  VARCHAR(255),
    created_at  TIMESTAMP      NOT NULL DEFAULT now()
);
```

```
-- ----- 7 · Índices de consulta -----
```

```
CREATE INDEX ix_pagos_aportante    ON pagos (aportante_id);
CREATE INDEX ix_pagos_ejercicio    ON pagos (ejercicio_id);
CREATE INDEX ix_pagos_estado      ON pagos (estado);
CREATE INDEX ix_pagos_fecha       ON pagos (fecha);
CREATE INDEX ix_ver_pago          ON verificaciones (pago_id);
CREATE INDEX ix_mov_fondo_fecha    ON movimientos_fondo (fondo_id, created_at);
CREATE INDEX ix_aportantes_apenom ON aportantes (apellido, nombre);
CREATE INDEX ix_libretas_aportante ON libretas (aportante_id, ejercicio_id);
```

```
-- ----- 8 · Sólo un fondo de capital activo -----
```

```
CREATE UNIQUE INDEX uq_fondo_capital_activo
    ON fondos ((TRUE)) WHERE tipo = 'capital' AND activo;
```

B Anexo · Pruebas de las restricciones

El script del anexo A se ejecutó sobre PostgreSQL 16 y las quince tablas se crearon sin errores. Después se intentó cargar, a propósito, once filas inválidas. La base rechazó las once.

Esto es lo que quiere decir que el modelo se sostiene solo: ninguna de estas once situaciones necesita que la aplicación las controle, porque la base no las deja entrar.

#	QUÉ SE INTENTÓ CARGAR	RESTRICCIÓN QUE LO RECHAZÓ
1	Una segunda operación con el mismo código de transacción bancaria.	operaciones_codigo_transaccion_key
2	Un pago que entra por su operación bancaria y por un reparto grupal a la vez.	ck_pagos_origen
3	Un pago que no entra por ninguna de las dos vías.	ck_pagos_origen
4	Un socio asignado a 5° año, en una carrera que sólo dicta tres.	fk_aportantes_carrera_anio
5	Un aportante no socio, sin CUIT.	ck_aportantes_subtipo
6	Un segundo ejercicio vigente, con otro ya abierto.	uq_ejercicio_vigente
7	Una verificación con resultado rechazado, sin motivo.	ck_ver_motivo
8	Una operación con importe negativo.	ck_operaciones_importe
9	Un movimiento de fondo que declara origen = pago pero no dice de cuál.	ck_mov_coherencia
10	Una solicitud marcada como aprobada, sin importe aprobado ni fondo.	ck_sol_aprobada
11	Un segundo aportante con un DNI ya cargado.	aportantes_dni_key

Una comprobación más, chica pero significativa: la cuota se cargó como 15000.50 y al leerla de vuelta seguía siendo 15000.50. Con la columna declarada como int, ese medio peso se habría perdido en el momento de guardarlo.

El archivo pruebas.sql, entregado junto con este documento, contiene las dos partes: los datos válidos y los once intentos que deben fallar.